
Figure 2: PoseNet estimation sample output

```
// Retrain the network
classifier.train(function(lossValue) {
  console.log("Loss is", lossValue);
});

// Get a prediction for that image
classifier.classify(document.
getElementById("dogB"), function(err,
result) {
  console.log(result); // Should output
'dog'
});
```

You will have noticed that the steps are simple. A live demo with the code and output is available at *https://editor.p5js.org/ml5/sketches/FeatureExtractor_Image_Regression.*

## Human pose estimation

The complicated tasks such as human pose estimation can be performed with Ml5.js with simple steps. The Ml5. poseNet code fragment is shown below:

```
const video = document.
getElementById("video");

// Create a new poseNet method
const poseNet = ml5.poseNet(video,
modelLoaded);

// When the model is loaded
function modelLoaded() {
  console.log("Model Loaded!");
}
// Listen to new 'pose' events
poseNet.on("pose", function(results) {
  poses = results;
});
```

A live demo of pose estimation is available at *https://editor.p5js.org/ml5/sketches/PoseNet_image_single*. A sample output is shown in Figure 2.

## Sound classification

The *ml5.soundClassifier* method can be used to classify pre-defined sounds. The following example uses the model *SpeechCommands18w*. This model can be used to detect speech commands for digits 'Zero'to 'Nine'. Apart from this, certain classes such as 'down', 'left' and ' right' can be detected with this model.

```
// Options for the SpeechCommands18w
model, the default probabilityThreshold
is 0
const options = { probabilityThreshold:
0.7 };
const classifier = ml5.soundClassi
fier('SpeechCommands18w', options,
modelReady);

function modelReady() {
  // classify sound
  classifier.classify(gotResult);
}
function gotResult(error, result) {
  if (error) {
    console.log(error);
    return;
  }
  // log the result
  console.log(result);
}
```

## Sentiment analysis

The *ml5.sentiment()* method is used to predict the sentiment of any given text. The following code fragment uses the *moviereviews* model:

```
// Create a new Sentiment method
const sentiment = ml5.
sentiment('movieReviews', modelReady);

// When the model is loaded
function modelReady() {
  // model is ready
  console.log("Model Loaded!");
}

// make the prediction
const prediction = sentiment.
predict(text);
console.log(prediction);
```

A live interactive demo is available at *https://editor.p5js.org/ml5/sketches/Sentiment_Interactive.*

The official documentation (*https://learn.ml5js.org/docs/#/*) has detailed information about the features of Ml5.js as well as links to live working demos of various use cases. If you are trying to incorporate ML abilities into a Web application, then you should explore Ml5.js. END

References

[1] *https://ml5js.org/*
[2] *https://learn.ml5js.org/docs/#/reference/index*

By: Dr K.S. Kuppusamy

The author is assistant professor of computer science, School of Engineering and Technology, Pondicherry Central University. He has 14+ years of teaching and research experience in academia and industry. His research interests include accessible computing, Web information retrieval, mobile computing, etc.